



Università degli Studi di Bergamo
Dipartimento di Ingegneria

Corso di Laurea in Ingegneria Informatica

Anno Accademico 2025/2026

MAINTENANCE

Ingegneria del Software

The Scouting Suite

Brivio Andrea — Matricola: 1089359

Fischetti Alessandro — Matricola: 1080491

Pesenti Lorenzo — Matricola: 1072635

Indice

1	Introduzione	2
1.1	Scopo del Documento	2
1.2	Contesto del Progetto	2
2	Tipologie di Manutenzione	2
3	Attività di Refactoring	3
3.1	Refactoring Architetturale	3
3.2	Introduzione Design Pattern	3
3.3	Correzioni Identificate da SonarLint	4
3.4	Correzioni Identificate da StanIDE	4
4	Gestione dei Cambiamenti	4
4.1	Strumenti Utilizzati	4
4.2	Procedure di Gestione Cambiamenti	5
4.3	Versionamento	5
5	Attività Future di Manutenzione	6
5.1	Priorità Alta (Entro 6 mesi)	6
5.2	Priorità Media (Entro 1 anno)	6
5.3	Priorità Bassa (Roadmap)	6
6	Registro delle Modifiche	7
6.1	Metriche di Qualità Post-Refactoring	7
7	Conclusioni	7
7.1	Risultati Raggiunti	7
7.2	Lezioni Apprese	8
7.3	Piano di Manutenzione Continuativa	8

1 Introduzione

1.1 Scopo del Documento

Questo documento descrive le strategie, le attività e le linee guida per la manutenzione del software **The Scouting Suite**, con particolare attenzione al miglioramento continuo della qualità del codice e dell'architettura attraverso attività di refactoring e l'introduzione di design pattern.

1.2 Contesto del Progetto

The Scouting Suite è un'applicazione Spring Boot + Vaadin per l'analisi statistica avanzata di giocatori di calcio, caratterizzata da:

- Architettura a 3 layer (Presentation, Business, Persistence)
- Gestione di 100+ attributi statistici per giocatore
- Sistema di filtraggio dinamico tramite pattern Specification
- Interfaccia utente configurabile con Vaadin

2 Tipologie di Manutenzione

1. Manutenzione Correttiva

- Risoluzione di bug rilevati durante il test e l'uso dell'applicazione.
- **Esempi concreti:**
 - Correzione parsing valori nulli/NaN nel CSV
 - Fix conversione tipi dati nei filtri statistici
 - Gestione eccezioni in PlayerSpecificationFactory

2. Manutenzione Adattativa

- Aggiornamenti per garantire compatibilità con:
 - Spring Boot 3.x → 3.2+
 - Java 17 → 21
 - Vaadin 24 → 24.x
 - Dipendenze Maven (JPA, H2, Tablesaw)

3. Manutenzione Perfettiva

- Ottimizzazione performance query dinamiche
- Miglioramento UI/UX (Grid configurabile, filtri in tempo reale)
- Refactoring DataProcessingService per gestione errori robusta
- Implementazione caching per dati statistici frequenti

4. Manutenzione Preventiva

- Refactoring guidato da SonarLint/StamIDE

- Introduzione pattern per estendibilità
- Miglioramento documentazione Javadoc
- Aumento code coverage testing

3 Attività di Refactoring

Le attività di refactoring sono state guidate da **SonarLint** e **StanIDE**, strumenti per l'analisi statica del codice, che hanno permesso di identificare e correggere problematiche legate alla qualità del software.

3.1 Refactoring Architetturale

- **Separazione layer:** Suddivisione chiara in Presentation (UI), Business (Services), Persistence (Repository)
- **Inversione dipendenze:** Application → Controller → Service → Repository
- **Riorganizzazione package:** struttura logica `com.scouting.[ui|service|data]`

3.2 Introduzione Design Pattern

- **Strategy Pattern** (StatQueryStrategy):
 - Separazione algoritmi filtraggio per tipo dato
 - Estendibilità nuovi tipi di filtro senza modificare codice esistente
- **Factory Pattern** (PlayerSpecificationFactory):
 - Creazione dinamica oggetti Specification per query JPA
 - Centralizzazione logica costruzione criteri
- **Specification Pattern:**
 - Implementazione JPA Specifications per query complesse
 - Composizione criteri AND/OR
- **Repository Pattern:**
 - Astrazione accesso dati tramite Spring Data JPA
 - Separazione logica business dalla persistenza
- **DTO Pattern** (PlayerFilterRequest, StatFilterCriteria):
 - Oggetti immutabili per trasferimento dati tra layer
 - Riduzione accoppiamento componenti

3.3 Correzioni Identificate da SonarLint

- **Codice duplicato:**
 - Unificazione metodi getInt/getDouble/getString in DataProcessingService
 - Rimozione logica ripetuta in MainView per configurazione colonne
- **Complessità ciclomatica:**
 - Suddivisione metodo saveToDatabase() in DataProcessingService
 - Estrazione logica costruzione Player in metodi helper
- **Possibili null pointer:**
 - Aggiunta controlli null in PlayerSpecificationFactory
 - Validazione input in MainView.updateList()

3.4 Correzioni Identificate da StanIDE

- **Struttura modulare:**
 - Analisi dipendenze tra package con JDepend
 - Riduzione afferent coupling per classi di basso livello
 - Aumento instabilità controllata per layer alti (UI)
- **Ottimizzazione dipendenze:**
 - Rimozione dipendenze cicliche tra package
 - Uso Dependency Injection per testing

4 Gestione dei Cambiamenti

4.1 Strumenti Utilizzati

- **SonarLint:** Analisi qualità codice in IDE (Eclipse)
- **StanIDE:** Valutazione metriche complessità e struttura
- **GitHub:** Controllo versione con GitFlow semplificato
- **GitHub Issues:** Tracciamento bug e feature request
- **JUnit 5 + Mockito:** Testing automatico

4.2 Procedure di Gestione Cambiamenti

1. Identificazione:

- Issue GitHub con descrizione dettagliata
- Analisi impatto (componenti coinvolti)
- Stima sforzo sviluppo

2. Pianificazione:

- Assegnazione issue a sviluppatore
- Creazione branch feature/nome-issue
- Definizione criteri di accettazione

3. Sviluppo:

- Implementazione modifiche in branch isolato
- Test unitari e di integrazione
- Documentazione modifiche

4. Verifica:

- Code review tramite Pull Request
- Analisi SonarLint post-modifica
- Test manuali su casi d'uso critici

5. Integrazione:

- Merge sul branch develop dopo approvazione
- Tagging versione su main per release

4.3 Versionamento

- **Schema:** Semantic Versioning (MAJOR.MINOR.PATCH)
- **Branch:**
 - **main:** codice stabile, solo release
 - **develop:** integrazione feature
 - **feature/*:** sviluppo nuove funzionalità
 - **hotfix/*:** correzioni urgenti
- **Convenzioni commit:** Conventional Commits

5 Attività Future di Manutenzione

5.1 Priorità Alta (Entro 6 mesi)

1. Miglioramento Performance:

- Implementazione paginazione risultati (attualmente tutti in memoria)
- Caching query frequenti con Spring Cache
- Ottimizzazione indici database per filtri statistici

2. Estensione Filtri:

- Aggiunta filtri complessi (percentili, deviazione standard)
- Supporto espressioni logiche avanzate (AND/OR annidati)
- Filtri temporali (performance per periodo specifico)

3. Testing:

- Aumento code coverage >90%
- Test end-to-end con TestContainers
- Performance testing query complesse

5.2 Priorità Media (Entro 1 anno)

1. Miglioramento UI/UX:

- Dashboard con grafici statistici (Chart.js integration)
- Salvataggio preset filtri
- Esportazione avanzata (PDF con grafici, Excel personalizzato)

2. Architettura:

- Modularizzazione in microservizi (separazione UI/API/Processing)
- Implementazione API REST per integrazione sistemi esterni
- Containerizzazione con Docker

5.3 Priorità Bassa (Roadmap)

1. Funzionalità Avanzate:

- Machine Learning per talent identification
- Analisi predittiva performance
- Social features per community scout
- App mobile companion

6 Registro delle Modifiche

Data	Descrizione della Modifica	Motivazione	Esito
20/12/2025	Refactoring codice con SonarLint	Migliorare qualità del codice	Implementato
07/01/2026	Riorganizzazione package struttura	Aumentare modularità e testabilità	Implementato
10/01/2026	Introduzione Pattern Specification	Gestire query dinamiche complesse	Implementato
15/01/2026	Refactoring DataProcessingService	Ridurre complessità e migliorare gestione errori	Implementato
20/01/2026	Implementazione Strategy Pattern	Separare algoritmi filtraggio per tipo dato	Implementato
25/01/2026	Ottimizzazione PlayerSpecificationFactory	Ridurre cognitive complexity	Implementato
30/01/2026	Aggiunta test integrazione ScoutingService	Aumentare code coverage e affidabilità	Implementato
05/02/2026	Miglioramento gestione dipendenze Spring	Facilitare testing e inversion of control	Implementato
10/02/2026	Refactoring MainView (estrazione componenti)	Ridurre complessità UI e migliorare manutenzione	In corso
15/02/2026	Implementazione caching query frequenti	Migliorare performance sistema	Pianificato
20/02/2026	Aggiunta paginazione risultati	Gestire grandi dataset efficientemente	Pianificato

Tabella 1: Registro delle modifiche effettuate durante la manutenzione

6.1 Metriche di Qualità Post-Refactoring

Metrica	Pre-Refactoring	Post-Refactoring
Code Coverage	65%	82%
Cognitive Complexity (max)	45	18
Duplicated Lines	8.5%	2.1%
Maintainability Index	68	85
SonarLint Issues	47	12
Test Pass Rate	88%	96%

Tabella 2: Confronto metriche di qualità prima e dopo le attività di manutenzione

7 Conclusioni

Attraverso una gestione strutturata delle modifiche e attività di refactoring mirate, il software **The Scouting Suite** è stato significativamente migliorato sia in termini di qualità del codice che di architettura.

7.1 Risultati Raggiunti

- **Architettura solida:** Clean separation of concerns con layer ben definiti
- **Codice mantenibile:** Pattern appropriati, bassa complessità, alta testabilità
- **Performance:** Query ottimizzate, gestione efficiente dati
- **Estendibilità:** Nuove feature aggiungibili senza impattare codice esistente

7.2 Lezioni Apprese

- L'analisi statica (SonarLint/StamIDE) è fondamentale per identificare technical debt
- I design pattern risolvono problemi reali di complessità, non vanno applicati a priori
- Il testing automatizzato è essenziale per manutenzione sicura
- La documentazione aggiornata riduce il costo delle modifiche future

7.3 Piano di Manutenzione Continuativa

Il sistema implementa un ciclo di manutenzione continuativa che garantisce:

- **Affidabilità:** Testing automatizzato e monitoraggio
- **Estendibilità:** Architettura modulare e pattern appropriati
- **Adattabilità:** Processo di cambiamento strutturato
- **Sostenibilità:** Documentazione completa e codice leggibile

The Scouting Suite rappresenta un caso di studio valido di applicazione dei principi di Ingegneria del Software (Sommerville) in un progetto reale, dimostrando come manutenzione proattiva e refactoring guidato da metriche possano garantire la longevità e l'evoluzione di un sistema software complesso.